Session 9 — Finish the Stocks tab

In class (90 min). Type along with the instructor. The project is **not graded**. The final exam is a closed-book MCQ: you must be able to **explain every line** you type.

Goal. Drive the chart with lookback / dates and show trailing and calendar returns.

You leave with. A complete S&P 500 **snapshot** (one index, not a watchlist). This is the first half of the minimum path.

Start Streamlit from last week:

```
uv run streamlit run dashboard.py
```

0.1 What we are not building

No ticker search, no many stocks on one chart, no download button. One series: `^GSPC`.

0.2 Step 1 — Date helpers in `views/common.py`

Add near the top (after the imports), with `DEFAULT_END_DATE`:

```
LOOKBACK_YEARS = {
    "1y": 1,
    "3y": 3,
    "5y": 5,
    "10y": 10,
    "20y": 20,
    "30y": 30,
    "max": None,
}


def lookback_start(end: date, years: int | None, earliest: date) -> date:
    if years is None:
        return earliest
    try:
        start = end.replace(year=end.year - years)
    except ValueError:
        start = end.replace(year=end.year - years, month=2, day=28)
    return max(earliest, start)


def date_range_error(start_date: date, end_date: date) -> str | None:
    if start_date > end_date:
        return "Start date must be on or before the end date."
    if start_date == end_date:
        return "Date range must span at least two distinct dates."
    return None
```

> **Note.** `except ValueError` handles 29 February minus one year in a non-leap year (for example 2024-02-29 → 2023-02-28).

0.3 Step 2 — Return tables in `views/common.py`

Add these functions. They will be reused on Commodities (stretch).

```
def _simple_return(last: float, base: float) -> float | None:
    if base == 0:
        return None
    return float(last / base - 1)


def _trailing_return(series: pd.Series, days: int) -> float | None:
```

```
    series = series.dropna().sort_index()
    if series.empty:
        return None
    last = series.iloc[-1]
    prior = series.loc[: series.index[-1] - pd.Timedelta(days=days)]
    if prior.empty:
        return None
    return _simple_return(last, prior.iloc[-1])


def _since_return(series: pd.Series, period_start: pd.Timestamp) -> float | None:
    series = series.dropna().sort_index()
    window = series.loc[period_start:]
    if window.empty:
        return None
    return _simple_return(window.iloc[-1], window.iloc[0])


def _format_return(value: float | None) -> str:
    if value is None:
        return "—"
    return f"{value:+.1%}"


def compute_return_metrics(prices: pd.Series):
    series = prices.dropna()
    if series.empty:
        return None
    last_day = series.index[-1]
    week_start = last_day - pd.Timedelta(days=int(last_day.weekday()))
    month_start = last_day.replace(day=1)
    quarter_month = ((last_day.month - 1) // 3) * 3 + 1
    quarter_start = last_day.replace(month=quarter_month, day=1)
    year_start = last_day.replace(month=1, day=1)
    trailing = {
        "7-day": _trailing_return(prices, 7),
        "30-day": _trailing_return(prices, 30),
        "90-day": _trailing_return(prices, 90),
        "1-year": _trailing_return(prices, 365),
    }
    calendar = {
        "Week to date": _since_return(prices, week_start),
        "Month to date": _since_return(prices, month_start),
        "Quarter to date": _since_return(prices, quarter_start),
        "Year to date": _since_return(prices, year_start),
    }
    return trailing, calendar


def render_return_metrics(prices: pd.Series) -> None:
    metrics = compute_return_metrics(prices)
    if metrics is None:
        st.info("No prices available to compute returns.")
        return
    trailing, calendar = metrics
    st.caption("Trailing returns")
    st.dataframe(
        pd.DataFrame([trailing], index=["Return"]).map(
            lambda v: _format_return(v) if not isinstance(v, str) else v
        ),
        use_container_width=True,
        hide_index=True,
    )
    st.caption("Calendar returns")
    st.dataframe(
        pd.DataFrame([calendar], index=["Return"]).map(
```

```
            lambda v: _format_return(v) if not isinstance(v, str) else v
        ),
        use_container_width=True,
        hide_index=True,
    )
```

If `DataFrame.map` fails on an older pandas, use `.apply(lambda col: col.map(...))` with the instructor.

The instructor version colours HTML tables. `st.dataframe` is enough if you can explain the return math.

0.4 Step 3 — Sidebar on the Stocks page

Rewrite `render()` in `views/equities.py` so widgets live in `st.sidebar`. Widget **keys** (`lookback`, `start_date`, `end_date`) must be unique on this page.

Fetch more history than you plot. One-year trailing return needs about 365 calendar days **before** the last price, even if the chart shows only 2024–2025.

```
from datetime import timedelta

import pandas as pd
import plotly.graph_objects as go
import streamlit as st

from views.common import (
    DEFAULT_END_DATE,
    DEFAULT_START_DATE,
    LOOKBACK_YEARS,
    chart_layout,
    date_range_error,
    download_data,
    get_available_date_bounds,
    lookback_start,
    preprocess,
    render_return_metrics,
)

SP500_TICKER = "^GSPC"


def render() -> None:
    st.header("Stocks")

    bounds = get_available_date_bounds(SP500_TICKER)
    earliest_date = bounds[0] if bounds else DEFAULT_START_DATE

    def _apply_lookback() -> None:
        end = DEFAULT_END_DATE
        years = LOOKBACK_YEARS[st.session_state.lookback]
        st.session_state.end_date = end
        st.session_state.start_date = lookback_start(end, years, earliest_date)

    if "lookback" not in st.session_state:
        st.session_state.lookback = "1y"
    if "start_date" not in st.session_state or "end_date" not in st.session_state:
        _apply_lookback()

    with st.sidebar:
        st.selectbox(
            "Lookback",
            list(LOOKBACK_YEARS),
            key="lookback",
            on_change=_apply_lookback,
        )
        start_date = st.date_input(
            "Start date",
```

```
            min_value=earliest_date,
            max_value=DEFAULT_END_DATE,
            key="start_date",
        )
        end_date = st.date_input(
            "End date",
            min_value=earliest_date,
            max_value=DEFAULT_END_DATE,
            key="end_date",
        )

    err = date_range_error(start_date, end_date)
    if err:
        st.warning(err)
        return

    metrics_start = max(earliest_date, end_date - timedelta(days=365 + 21))
    fetch_start = min(start_date, metrics_start)
    prices = download_data((SP500_TICKER,), fetch_start, end_date, use_live=True)
    prices = preprocess(prices)
    if prices.empty:
        st.error("No S&P 500 data.")
        return

    chart_mask = (prices.index >= pd.Timestamp(start_date)) & (
        prices.index <= pd.Timestamp(end_date)
    )
    chart_prices = prices.loc[chart_mask]
    if chart_prices.empty:
        st.warning("No observations in the selected date range.")
        return

    bundled = get_available_date_bounds(SP500_TICKER)
    if bundled is not None and prices.index[-1].date() <= bundled[1] < end_date:
        st.info("Showing bundled S&P 500 data (live Yahoo fetch unavailable).")

    st.sidebar.caption(f"Last updated: {prices.index[-1].strftime('%Y-%m-%d')}")
    render_return_metrics(prices[SP500_TICKER])

    series = chart_prices[SP500_TICKER]
    fig = go.Figure()
    fig.add_trace(go.Scatter(x=series.index, y=series, mode="lines", name="S&P 500"))
    fig.update_layout(
        **chart_layout(
            title="S&P 500",
            height=450,
            yaxis_title="Index level",
            xaxis_title="Date",
            hovermode="x unified",
        )
    )
    st.plotly_chart(fig, use_container_width=True)


render()
```

0.5 Step 4 — Check

- Changing Lookback to 5y moves the start date.
- Trailing **1-year** is not empty on a 1y chart (because `fetch_start` goes further back).
- Invalid range (start after end) shows a warning, not a crash.

0.6 Step 5 — Commit

```
git add views/common.py views/equities.py
git commit -m "session 9: equities snapshot"
```

0.7 Next week

Bonds: FRED yields, **new** sidebar keys `bonds_lookback`, `bonds_start_date`, `bonds_end_date` so they do not clash with Stocks.